

04.类和对象(下)

1. 再探构造函数

- 之前我们实现构造函数时，初始化成员变量主要使用函数体内赋值，构造函数初始化还有一种方式，就是初始化列表，初始化列表的使用方式是以一个冒号开始，接着是一个以逗号分隔的数据成员列表，每个"成员变量"后面跟一个放在括号中的初始值或表达式。
- 每个成员变量在初始化列表中只能出现一次，语法理解上初始化列表可以认为是每个成员变量定义初始化的地方。
- 引用成员变量，const成员变量，没有默认构造的类类型变量，必须放在初始化列表位置进行初始化，否则会编译报错。
- C++11支持在成员变量声明的位置给缺省值，这个缺省值主要是给没有显示在初始化列表初始化的成员使用的。
- 尽量使用初始化列表初始化，因为那些你不在初始化列表初始化的成员也会走初始化列表，如果这个成员在声明位置给了缺省值，初始化列表会用这个缺省值初始化。如果你没有给缺省值，对于没有显示在初始化列表初始化的内置类型成员是否初始化取决于编译器，C++并没有规定。对于没有显示在初始化列表初始化的自定义类型成员会调用这个成员类型的默认构造函数，如果没有默认构造会编译错误。
- 初始化列表中按照成员变量在类中声明顺序进行初始化，跟成员在初始化列表出现的先后顺序无关。建议声明顺序和初始化列表顺序保持一致。

初始化列表总结：

无论是否显示写初始化列表，每个构造函数都有初始化列表；

无论是否在初始化列表显示初始化成员变量，每个成员变量都要走初始化列表初始化；

成员变量走初始化列表的逻辑

1. 显示在初始化列表初始化的成员变量就按这个值初始化

2. 未显示在初始化列表的成员变量

2.1 类中声明位置有缺省值，就按缺省值初始化

2.2 类中声明位置没有缺省值

内置类型成员变量，可能初始化为随机值，也可能是默认值0等，具体看编译器

自定义类型成员变量，调用这个成员的默认构造函数

引用成员变量/const成员变量/没有默认构造函数的成员变量必须在初始化列表显示初始化或者未显示初始化但是声明时有缺省值，否则编译报错

```
1  #include<iostream>
2  using namespace std;
3
4  class Time
5  {
6  public:
7      Time(int hour)
8          :_hour(hour)
9      {
10         cout << "Time()" << endl;
11     }
12 private:
13     int _hour;
14 };
15
16 class Date
17 {
18 public:
19     Date(int& x, int year = 1, int month = 1, int day = 1)
20         :_year(year)
21         ,_month(month)
22         ,_day(day)
23         ,_t(12)
24         ,_ref(x)
25         ,_n(1)
26     {
27         // error C2512: "Time": 没有合适的默认构造函数可用
28         // error C2530 : "Date::_ref" : 必须初始化引用
29         // error C2789 : "Date::_n" : 必须初始化常量限定类型的对象
30     }
31
32     void Print() const
```

```

33     {
34         cout << _year << "-" << _month << "-" << _day << endl;
35     }
36 private:
37     int _year;
38     int _month;
39     int _day;
40
41     Time _t;           // 没有默认构造
42     int& _ref;         // 引用
43     const int _n;     // const
44 };
45
46 int main()
47 {
48     int i = 0;
49     Date d1(i);
50     d1.Print();
51
52     return 0;
53 }

```

```

1  #include<iostream>
2  using namespace std;
3
4  class Time
5  {
6  public:
7      Time(int hour)
8          :_hour(hour)
9      {
10         cout << "Time()" << endl;
11     }
12 private:
13     int _hour;
14 };
15
16 class Date
17 {
18 public:
19     Date()
20         :_month(2)
21     {
22         cout << "Date()" << endl;
23     }

```

```

24
25     void Print() const
26     {
27         cout << _year << "-" << _month << "-" << _day << endl;
28     }
29 private:
30     // 注意这里不是初始化，这里给的是缺省值，这个缺省值是给初始化列表的
31     // 如果初始化列表没有显示初始化，默认就会用这个缺省值初始化
32     int _year = 1;
33     int _month = 1;
34     int _day;
35
36     Time _t = 1;
37     const int _n = 1;
38     int* _ptr = (int*)malloc(12);
39 };
40
41 int main()
42 {
43     Date d1;
44     d1.Print();
45
46     return 0;
47 }

```

下面程序的运行结果是什么（）

- A. 输出 1 1
- B. 输出 2 2
- C. 编译报错
- D. 输出 1 随机值
- E. 输出 1 2
- F. 输出 2 1

```

1  #include<iostream>
2  using namespace std;
3
4  class A
5  {
6  public:
7      A(int a)
8          :_a1(a)
9          , _a2(_a1)

```

```

10     {}
11
12     void Print() {
13         cout << _a1 << " " << _a2 << endl;
14     }
15 private:
16     int _a2 = 2;
17     int _a1 = 2;
18 };
19
20 int main()
21 {
22     A aa(1);
23     aa.Print();
24 }

```

2. 类型转换

- C++支持内置类型隐式类型转换为类类型对象，需要有相关内置类型为参数的构造函数。
- 构造函数前面加explicit就不再支持隐式类型转换。
- 类类型的对象之间也可以隐式转换，需要相应的构造函数支持。

```

1  #include<iostream>
2  using namespace std;
3
4  class A
5  {
6  public:
7      // 构造函数explicit就不再支持隐式类型转换
8      // explicit A(int a1)
9      A(int a1)
10         :_a1(a1)
11     {}
12
13     //explicit A(int a1, int a2)
14     A(int a1, int a2)
15         :_a1(a1)
16         , _a2(a2)
17     {}
18
19     void Print()
20     {
21         cout << _a1 << " " << _a2 << endl;
22     }

```

```

23
24     int Get() const
25     {
26         return _a1 + _a2;
27     }
28 private:
29     int _a1 = 1;
30     int _a2 = 2;
31 };
32
33 class B
34 {
35 public:
36     B(const A& a)
37         :_b(a.Get())
38     {}
39
40 private:
41     int _b = 0;
42 };
43
44 int main()
45 {
46     // 1构造一个A的临时对象，再用这个临时对象拷贝构造aa3
47     // 编译器遇到连续构造+拷贝构造->优化为直接构造
48     A aa1 = 1;
49     aa1.Print();
50
51     const A& aa2 = 1;
52
53     // C++11之后才支持多参数转化
54     A aa3 = { 2,2 };
55
56     // aa3隐式类型转换为b对象
57     // 原理跟上面类似
58     B b = aa3;
59     const B& rb = aa3;
60
61     return 0;
62 }

```

3. static成员

- 用static修饰的成员变量，称之为静态成员变量，静态成员变量一定要在类外进行初始化。
- 静态成员变量为所有类对象所共享，不属于某个具体的对象，不存在对象中，存放在静态区。

- 用static修饰的成员函数，称之为静态成员函数，静态成员函数没有this指针。
- 静态成员函数中可以访问其他的静态成员，但是不能访问非静态的，因为没有this指针。
- 非静态的成员函数，可以访问任意的静态成员变量和静态成员函数。
- 突破类域就可以访问静态成员，可以通过类名::静态成员 或者 对象.静态成员 来访问静态成员变量和静态成员函数。
- 静态成员也是类的成员，受public、protected、private 访问限定符的限制。
- 静态成员变量不能在声明位置给缺省值初始化，因为缺省值是个构造函数初始化列表的，静态成员变量不属于某个对象，不走构造函数初始化列表。

```
1 // 实现一个类，计算程序中创建出了多少个类对象？
2 #include<iostream>
3 using namespace std;
4
5 class A
6 {
7 public:
8     A()
9     {
10         ++_scount;
11     }
12
13     A(const A& t)
14     {
15         ++_scount;
16     }
17
18     ~A()
19     {
20         --_scount;
21     }
22
23     static int GetACount()
24     {
25         return _scount;
26     }
27 private:
28     // 类里面声明
29     static int _scount;
30 };
31
32 // 类外面初始化
33 int A::_scount = 0;
34
```

```

35 int main()
36 {
37     cout << A::GetACount() << endl;
38     A a1, a2;
39     A a3(a1);
40     cout << A::GetACount() << endl;
41     cout << a1.GetACount() << endl;
42
43     // 编译报错: error C2248: "A::_scount": 无法访问 private 成员(在“A”类中声明)
44     //cout << A::_scount << endl;
45
46     return 0;
47 }

```

求1+2+3+...+n_牛客题霸_牛客网

```

1 class Sum
2 {
3 public:
4     Sum()
5     {
6         _ret += _i;
7         ++_i;
8     }
9
10    static int GetRet()
11    {
12        return _ret;
13    }
14 private:
15     static int _i;
16     static int _ret;
17 };
18
19 int Sum::_i = 1;
20 int Sum::_ret = 0;
21
22 class Solution {
23 public:
24     int Sum_Solution(int n) {
25         // 变长数组
26         Sum arr[n];
27         return Sum::GetRet();
28     }
29 };

```


4. 友元

- 友元提供了一种突破类访问限定符封装的方式，友元分为：友元函数和友元类，在函数声明或者类声明的前面加friend，并且把友元声明放到一个类的里面。
- 外部友元函数可访问类的私有和保护成员，友元函数仅仅是一种声明，他不是类的成员函数。
- 友元函数可以在类定义的任何地方声明，不受类访问限定符限制。
- 一个函数可以是多个类的友元函数。
- 友元类中的成员函数都可以是另一个类的友元函数，都可以访问另一个类中的私有和保护成员。
- 友元类的关系是单向的，不具有交换性，比如A类是B类的友元，但是B类不是A类的友元。
- 友元类关系不能传递，如果A是B的友元，B是C的友元，但是A不是C的友元。
- 有时提供了便利。但是友元会增加耦合度，破坏了封装，所以友元不宜多用。

```
1  #include<iostream>
2  using namespace std;
3
4  // 前置声明，都则A的友元函数声明编译器不认识B
5  class B;
6
7  class A
8  {
9      // 友元声明
10     friend void func(const A& aa, const B& bb);
11 private:
12     int _a1 = 1;
13     int _a2 = 2;
14 };
15
16 class B
17 {
18     // 友元声明
19     friend void func(const A& aa, const B& bb);
20 private:
21     int _b1 = 3;
22     int _b2 = 4;
23 };
24
25 void func(const A& aa, const B& bb)
26 {
27     cout << aa._a1 << endl;
28     cout << bb._b1 << endl;
29 }
```

```
30
31 int main()
32 {
33     A aa;
34     B bb;
35     func(aa, bb);
36
37     return 0;
38 }
```

```
1 #include<iostream>
2 using namespace std;
3
4
5 class A
6 {
7     // 友元声明
8     friend class B;
9 private:
10     int _a1 = 1;
11     int _a2 = 2;
12 };
13
14 class B
15 {
16 public:
17     void func1(const A& aa)
18     {
19         cout << aa._a1 << endl;
20         cout << _b1 << endl;
21     }
22
23     void func2(const A& aa)
24     {
25         cout << aa._a2 << endl;
26         cout << _b2 << endl;
27     }
28 private:
29     int _b1 = 3;
30     int _b2 = 4;
31 };
32
33
34 int main()
35 {
```

```

36     A aa;
37     B bb;
38     bb.func1(aa);
39     bb.func1(aa);
40
41     return 0;
42 }

```

5. 内部类

- 如果一个类定义在另一个类的内部，这个内部类就叫做内部类。内部类是一个独立的类，跟定义在全局相比，他只是受外部类类域限制和访问限定符限制，所以外部类定义的对象中不包含内部类。
- 内部类默认是外部类的友元类。
- 内部类本质也是一种封装，当A类跟B类紧密关联，A类实现出来主要就是给B类使用，那么可以考虑把A类设计为B的内部类，如果放到private/protected位置，那么A类就是B类的专属内部类，其他地方都用不了。

```

1  #include<iostream>
2  using namespace std;
3
4  class A
5  {
6  private:
7      static int _k;
8      int _h = 1;
9  public:
10     class B // B默认就是A的友元
11     {
12     public:
13         void foo(const A& a)
14         {
15             cout << _k << endl;           //OK
16             cout << a._h << endl;         //OK
17         }
18
19         int _b1;
20     };
21 };
22
23 int A::_k = 1;
24
25 int main()
26 {
27     cout << sizeof(A) << endl;

```

```

28
29     A::B b;
30
31     A aa;
32     b.foo(aa);
33
34     return 0;
35 }

```

求1+2+3+...+n_牛客题霸_牛客网

```

1
2 class Solution {
3     // 内部类
4     class Sum
5     {
6     public:
7         Sum()
8         {
9             _ret += _i;
10            ++_i;
11        }
12    };
13
14    static int _i;
15    static int _ret;
16 public:
17    int Sum_Solution(int n) {
18        // 变长数组
19        Sum arr[n];
20        return _ret;
21    }
22 };
23
24 int Solution::_i = 1;
25 int Solution::_ret = 0;

```

6. 匿名对象

- 用 类型(实参) 定义出来的对象叫做匿名对象，相比之前我们定义的 类型 对象名(实参) 定义出来的叫有名对象
- 匿名对象生命周期只在当前一行，一般临时定义一个对象当前用一下即可，就可以定义匿名对象。

```

1  class A
2  {
3  public:
4      A(int a = 0)
5          :_a(a)
6      {
7          cout << "A(int a)" << endl;
8      }
9
10     ~A()
11     {
12         cout << "~A()" << endl;
13     }
14 private:
15     int _a;
16 };
17
18 class Solution {
19 public:
20     int Sum_Solution(int n) {
21         //...
22         return n;
23     }
24 };
25
26 int main()
27 {
28     A aa1;
29
30     // 不能这么定义对象，因为编译器无法识别下面是一个函数声明，还是对象定义
31     //A aa1();
32
33     // 但是我们可以这么定义匿名对象，匿名对象的特点不用取名字，
34     // 但是他的生命周期只有这一行，我们可以看到下一行他就会自动调用析构函数
35     A();
36     A(1);
37
38     A aa2(2);
39
40     // 匿名对象在这样场景下就很好用，当然还有一些其他使用场景，这个我们以后遇到了再说
41     Solution().Sum_Solution(10);
42
43     return 0;
44 }

```

7. 对象拷贝时的编译器优化

- 现代编译器会为了尽可能提高程序的效率，在不影响正确性的情况下会尽可能减少一些传参和传返回值的过程中可以省略的拷贝。
- 如何优化C++标准并没有严格规定，各个编译器会根据情况自行处理。当前主流的相对新一点的编译器对于连续一个表达式步骤中的连续拷贝会进行合并优化，有些更新更"激进"的编译器还会进行跨行跨表达式的合并优化。
- linux下可以将下面代码拷贝到test.cpp文件，编译时用 `g++ test.cpp -fno-elide-constructors` 的方式关闭构造相关的优化。

```
1 #include<iostream>
2 using namespace std;
3
4 class A
5 {
6 public:
7     A(int a = 0)
8         :_a1(a)
9     {
10         cout << "A(int a)" << endl;
11     }
12
13     A(const A& aa)
14         :_a1(aa._a1)
15     {
16         cout << "A(const A& aa)" << endl;
17     }
18
19     A& operator=(const A& aa)
20     {
21         cout << "A& operator=(const A& aa)" << endl;
22
23         if (this != &aa)
24         {
25             _a1 = aa._a1;
26
27         }
28
29         return *this;
30     }
31
32     ~A()
33     {
34         cout << "~A()" << endl;
35     }
36 private:
```

```

37         int _a1 = 1;
38     };
39
40     void f1(A aa)
41     {}
42
43     A f2()
44     {
45         A aa;
46         return aa;
47     }
48
49     int main()
50     {
51         // 传值传参
52         // 构造+拷贝构造
53         A aa1;
54         f1(aa1);
55         cout << endl;
56
57         // 隐式类型, 连续构造+拷贝构造->优化为直接构造
58         f1(1);
59
60         // 一个表达式中, 连续构造+拷贝构造->优化为一个构造
61         f1(A(2));
62         cout << endl;
63
64         cout << "*****" << endl;
65
66         // 传值返回
67         // 不优化的情况下传值返回, 编译器会生成一个拷贝返回对象的临时对象作为函数调用表达
        式的返回值
68
69         // 无优化 (vs2019 debug)
70         // 一些编译器会优化得更厉害, 将构造的局部对象和拷贝构造的临时对象优化为直接构造
        (vs2022 debug)
71         f2();
72         cout << endl;
73
74         // 返回时一个表达式中, 连续拷贝构造+拷贝构造->优化一个拷贝构造 (vs2019 debug)
75         // 一些编译器会优化得更厉害, 进行跨行合并优化, 将构造的局部对象aa和拷贝的临时对象
        和接收返回值对象aa2优化为一个直接构造。 (vs2022 debug)
76         A aa2 = f2();
77         cout << endl;
78
79         // 一个表达式中, 开始构造, 中间拷贝构造+赋值重载->无法优化 (vs2019 debug)

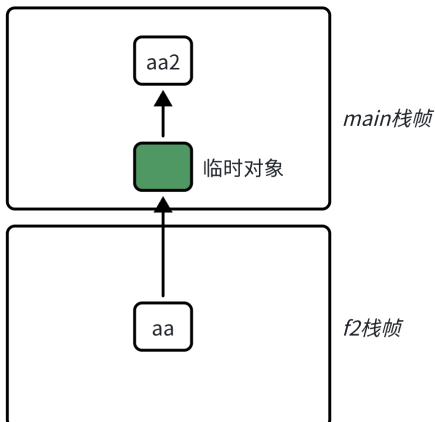
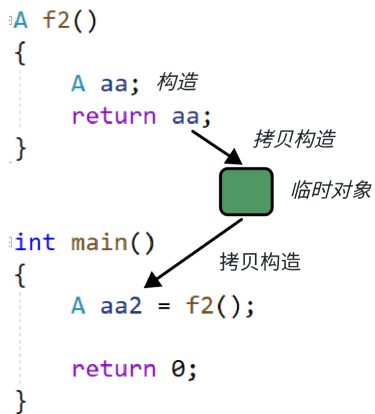
```

```

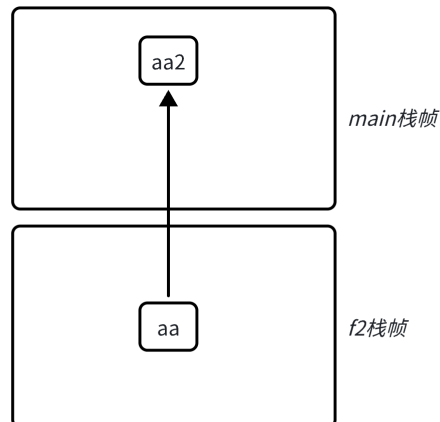
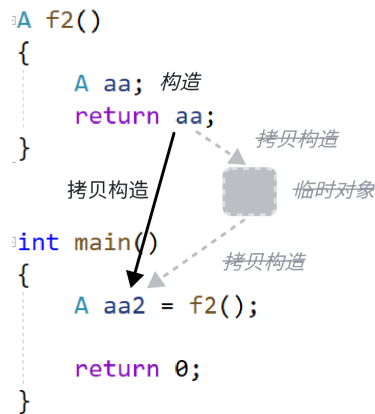
80      // 一些编译器会优化得更厉害，进行跨行合并优化，将构造的局部对象aa和拷贝临时对象合
      并为一个直接构造 (vs2022 debug)
81      aa1 = f2();
82      cout << endl;
83
84      return 0;
85  }

```

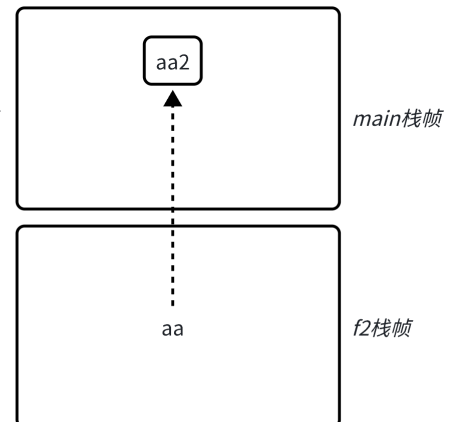
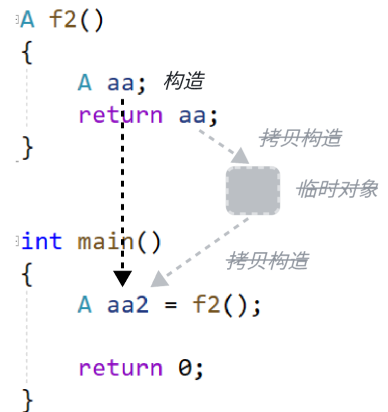
g++ test.cpp -fno-elide-constructors



vs2019 debug



vs2022 debug



底层实现的角度，没有构造aa
构造的是aa2，aa是aa2的引用